

ТЕОРИЯ ДЛЯ ПРЕЗЕНТАЦИИ ПО C++

Основы программирования

Основные темы: потоки ввода-вывода, форматирование данных, переменные, типы данных, литералы.

СОДЕРЖАНИЕ

1. Потоки ввода-вывода (cin, cout)
2. Форматирование вводимых данных
3. Форматирование выходных данных
4. Переменные и типы данных
5. Литералы
6. Практическое применение
7. Частые ошибки новичков
8. Полезные советы
9. Контрольные вопросы
10. Шпаргалка

ПОТОКИ ВВОДА-ВЫВОДА (CIN, COUT)

Определение потока

Поток (stream) — это последовательность байтов, передаваемых от источника к приемнику.

Типы потоков в C++

- cin — поток стандартного ввода (клавиатура)
- cout — поток стандартного вывода (экран)
- cerr — поток вывода ошибок
- clog — поток вывода сообщений

Подключение библиотеки

```
#include <iostream>  
using namespace std;
```

Оператор вывода cout

Синтаксис: cout << "данные";

Оператор << — оператор вставки (insertion operator)

Примеры

```
cout << "Привет, мир!";  
cout << 42;  
cout << "Число: " << 42;
```

ОПЕРАТОР ВВОДА CIN

Синтаксис:

```
cin >> переменная;
```

Оператор `>>` — оператор извлечения (extraction operator).

Примеры использования:

```
int age;  
cout << "Введите возраст: ";  
cin >> age;
```

```
string name;  
cout << "Введите имя: ";  
cin >> name;
```

⚠ **Важно!** `cin` при вводе строки останавливается на первом пробеле, табуляции или переносе строки.

Ввод нескольких значений:

```
int a, b, c;  
cin >> a >> b >> c; // Ввод: 1 2 3
```

getline для строк с пробелами:

```
string fullName;  
cout << "Введите полное имя: ";  
getline(cin, fullName); // Читает всю строку до Enter
```

ФОРМАТИРОВАНИЕ ВЫХОДНЫХ ДАННЫХ (ЧАСТЬ 1)

Подключение библиотеки `<iomanip>`:

```
#include <iomanip>
```

Основные манипуляторы:

1. `setw(n)` — установить ширину поля

```
cout << "|" << setw(10) << "Товар" << "|";  
// Вывод: |    Товар|
```

2. `setprecision(n)` — точность вывода дробных чисел

```
double pi = 3.14159265;  
cout << setprecision(3) << pi; // Вывод: 3.14
```

ФОРМАТИРОВАНИЕ ВЫХОДНЫХ ДАННЫХ (ЧАСТЬ 2)

3. `fixed` — фиксированный формат (для денежных значений)

```
double price = 199.9;  
cout << fixed;  
cout << setprecision(2) << price; // Вывод: 199.90
```

4. `scientific` — научный формат

```
double bigNumber = 123456789;  
cout << scientific;  
cout << bigNumber; // Вывод: 1.234568e+08
```

ФОРМАТИРОВАНИЕ ДАННЫХ (ЧАСТЬ 3)

`setfill('c')` — символ заполнения

```
cout << setfill('*') << setw(10) << 42;  
// Вывод: *****42
```

`left` и `right` — выравнивание

```
cout << left << setw(10) << "Товар"; // Товар____ (слева)  
cout << right << setw(10) << "Цена"; // ____Цена (справа)
```

ПРИМЕР КОМПЛЕКСНОГО ФОРМАТИРОВАНИЯ

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << "===== ЧЕК =====" << endl;
    cout << left << setw(20) << "Товар"
        << right << setw(10) << "Цена" << endl;
    cout << "-----" << endl;

    cout << left << setw(20) << "Молоко 1л"
        << right << setw(10) << fixed << setprecision(2) << 89.50 << endl;

    cout << left << setw(20) << "Хлеб"
        << right << setw(10) << fixed << setprecision(2) << 45.00 << endl;

    cout << "-----" << endl;
    cout << left << setw(20) << "ИТОГО:"
        << right << setw(10) << fixed << setprecision(2) << 134.50 << endl;

    return 0;
}
```


ПЕРЕМЕННЫЕ И ТИПЫ ДАННЫХ

Определение переменной

Переменная — это именованная область памяти для хранения данных.

Объявление переменных

Синтаксис:

```
тип_данных имя_переменной;  
тип_данных имя_переменной = значение; // Инициализация
```

Основные типы данных

- `int` — целое число
- `double` — число с плавающей точкой двойной точности
- `float` — число с плавающей точкой
- `char` — один символ
- `bool` — логическое значение `true/false`
- `string` — строка текста

ТИПЫ ДАННЫХ В ДЕТАЛЯХ (ЧАСТЬ 1)

Целочисленные типы:

- `short` — 2 байта (-32 768 до 32 767)
- `int` — 4 байта (-2 147 483 648 до 2 147 483 647)
- `long` — 4-8 байт
- `long long` — 8 байт ($\pm 9 \times 10^{18}$)

Типы с плавающей точкой:

- `float` — 4 байта (~6-7 знаков)
- `double` — 8 байт (~15-16 знаков)
- `long double` — 10-16 байт (~18-19 знаков)

Примеры кода:

```
int count = 100;  
short small = 1000;  
long big = 1000000;  
long long huge = 10000000000LL;  
  
float price = 19.99f;  
double pi = 3.14159265;  
long double precise = 3.14159265358979L;
```

ТИПЫ ДАННЫХ В ДЕТАЛЯХ (ЧАСТЬ 2)

Логический и символьный типы:

```
bool isReady = true;  
bool isError = false;  
  
char letter = 'A';  
char newline = '\n';
```

Строковый тип (требует `#include <string>`):

```
string name = "Иван";  
string greeting = "Привет!";
```

Примеры кода:

```
int count = 100;  
short small = 1000;  
long big = 1000000;  
long long huge = 10000000000LL;  
  
float price = 19.99f;  
double pi = 3.14159265;  
long double precise = 3.14159265358979L;
```

ПРАВИЛА ИМЕНОВАНИЯ ПЕРЕМЕННЫХ

Правила именования

Можно:

- Буквы латинского алфавита (a-z, A-Z)
- Цифры, но не в начале
- Символ подчеркивания _
- Регистр важен: age и Age — разные переменные

Нельзя:

- Начинать с цифры
- Использовать пробелы
- Использовать русские буквы
- Использовать ключевые слова C++ (int, double, if, while и т.д.)

СТИЛИ ИМЕНОВАНИЯ И КОНСТАНТЫ

Стили именования

camelCase

Рекомендуется в C++:

```
int studentAge;  
double averageScore;  
string firstName;
```

snake_case

```
int student_age;  
double average_score;
```

Константы

Константа — переменная, значение которой нельзя изменить после объявления.

const

Рекомендуется использовать этот способ.

```
const int MAX_SIZE = 100;  
const double PI = 3.14159;
```

#define

Старый стиль, не рекомендуется.

```
#define PI 3.14159  
#define MAX_SIZE 100
```

ЛИТЕРАЛЫ

Определение литерала

Литерал — это фиксированное значение, записанное непосредственно в коде.

Примеры

```
int age = 25;      // 25 — целочисленный литерал
double pi = 3.14;  // 3.14 — литерал с плавающей точкой
char grade = 'A';  // 'A' — символьный литерал
string name = "Иван"; // "Иван" — строковый литерал
bool flag = true;  // true — логический литерал
```

Целочисленные литералы

```
int decimal = 42;    // Десятичное: 42
int octal = 052;     // Восьмеричное (начинается с 0): 052 = 42
int hex = 0x2A;      // Шестнадцатеричное (начинается с 0x): 0x2A = 42
int binary = 0b101010; // Двоичное (C++14, начинается с 0b)
```

Суффиксы

```
unsigned int u = 42u;    // unsigned
long int l = 42L;        // long
long long int ll = 42LL;  // long long
unsigned long long ull = 42ULL; // unsigned long long
```

ЛИТЕРАЛЫ С ПЛАВАЮЩЕЙ ТОЧКОЙ

Литералы с плавающей точкой:

```
double d1 = 3.14;    // double по умолчанию  
float f1 = 3.14f;    // float (суффикс f или F)  
long double ld = 3.14L; // long double (суффикс l или L)
```

Научная запись:

```
double big = 1.23e10; //  $1.23 \times 10^{10}$   
double small = 5.67e-5; //  $5.67 \times 10^{-5}$ 
```

СИМВОЛЬНЫЕ И СТРОКОВЫЕ ЛИТЕРАЛЫ

Символьные литералы:

```
char a = 'A';      // Обычный символ
char newline = '\n'; // Перенос строки
char tab = '\t';   // Табуляция
char quote = '"';  // Одинарная кавычка
char backslash = '\\'; // Обратный слэш
char null = '\0';  // Нулевой символ
```

Строковые литералы:

```
string s1 = "Hello";
string s2 = "Hello\nWorld"; // С переносом строки
string s3 = "C:\\Users\\Admin"; // Путь в Windows
```

Raw string literals (C++11):

```
string path = R"(C:\Users\Documents\file.txt);
```

Логические литералы:

```
bool b1 = true;
bool b2 = false;
```


ПРАКТИЧЕСКИЙ ПРИМЕР: КАЛЬКУЛЯТОР

```
#include <iostream>
using namespace std;

int main() {
    double num1, num2, result;
    char operation;

    cin >> num1 >> operation >> num2;

    switch (operation) {
        case '+': result = num1 + num2; break;
        case '-': result = num1 - num2; break;
        case '*': result = num1 * num2; break;
        case '/': result = num1 / num2; break;
    }

    cout << result;
    return 0;
}
```

ПРАКТИЧЕСКИЙ ПРИМЕР: АНКЕТА СТУДЕНТА

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    setlocale(LC_ALL, "Russian");

    string fullName;
    int age;
    double averageGrade;
    char gender;
    bool isBudget;

    cout << "Введите ФИО: ";
    getline(cin, fullName);
    cout << "Введите возраст: ";
    cin >> age;
    cout << "Введите средний балл: ";
    cin >> averageGrade;
    cout << "Введите пол (м/ж): ";
    cin >> gender;
    cout << "Бюджетник? (1 - да, 0 - нет): ";
    cin >> isBudget;

    cout << "\n===== ИНФОРМАЦИЯ =====< endl;
    cout << left << setw(20) << "ФИО:" << right << setw(20) << fullName << endl;
    cout << left << setw(20) << "Возраст:" << right << setw(20) << age << endl;
    cout << left << setw(20) << "Средний балл:" << right << setw(20) << fixed << setprecision(2) << averageGrade << endl;
    cout << left << setw(20) << "Пол:" << right << setw(20) << gender << endl;
    cout << left << setw(20) << "Форма обучения:" << right << setw(20) << (isBudget ? "Бюджет" : "Контракт") << endl;

    return 0;
}
```

ПРАКТИЧЕСКИЙ ПРИМЕР: КОНВЕРТЕР ВАЛЮТ

```
#include <iostream>
using namespace std;

int main() {
    const double USD_TO_RUB = 92.50, EUR_TO_RUB = 100.75, CNY_TO_RUB = 12.80;
    double amount, rate;
    int choice;

    cout << "1. USD → RUB\n2. EUR → RUB\n3. CNY → RUB\nВыберите операцию: ";
    cin >> choice;
    cout << "Введите сумму: ";
    cin >> amount;

    switch (choice) {
        case 1: rate = USD_TO_RUB; break;
        case 2: rate = EUR_TO_RUB; break;
        case 3: rate = CNY_TO_RUB; break;
        default: cout << "Неверный выбор!"; return 1;
    }

    cout << "Результат: " << amount * rate << " RUB" << endl;
    return 0;
}
```

ЧАСТЫЕ ОШИБКИ НОВИЧКОВ (ЧАСТЬ 1)

Ошибки с cin/cout

Ошибка 1: Перепутаны стрелки

Неправильный код:

```
cout >> variable;  
cin << variable;
```

Правильный код:

```
cin >> variable;  
cout << variable;
```

Ошибка 2: Забыт endl или \n

Неправильный код:

```
cout << "Первая строка";  
cout << "Вторая строка";  
// Вывод: Первая строкаВторая строка
```

Правильный код:

```
cout << "Первая строка" << endl;  
cout << "Вторая строка" << endl;
```

Ошибка 3: Не очищен буфер после ввода числа

Неправильный код:

```
int age;  
string name;  
  
cin >> age; // Ввод: 25  
cin >> name; // name будет пустым!
```

Правильный код:

```
cin >> age;  
cin.ignore(); // Очистить буфер  
getline(cin, name);
```

ЧАСТЫЕ ОШИБКИ НОВИЧКОВ

ОШИБКИ С ПЕРЕМЕННЫМИ

Ошибка 1: Использование неинициализированной переменной

Неправильный код:

```
int x;  
cout << x; // Неопределенное поведение (мусор в памяти)
```

Правильный код:

```
int x = 0;  
cout << x;
```

Ошибка 2: Несоответствие типов

Неправильный код:

```
int number;  
cin >> number; // Пользователь ввел: 3.14  
// number будет равен 3 (дробная часть отброшена)
```

Правильный код:

```
double number;  
cin >> number;
```

ОШИБКИ И РЕШЕНИЯ (ЧАСТЬ 2)

Деление целых чисел

Неправильно

```
int a = 5;  
int b = 2;  
double result = a / b; // result = 2.0 (а не 2.5!)
```

Правильно

```
double result = static_cast<double>(a) / b; // 2.5  
// или:  
double result = 1.0 * a / b;
```

Ошибки форматирования

`setw` применяется не к тому

Неправильно

```
cout << setw(10) << "Товар" << "Цена";  
// "Цена" не будет выровнена!
```

Правильно

```
cout << setw(10) << "Товар" << setw(10) << "Цена";
```

Забыт `fixed` для денег

Неправильно

```
double price = 100.5;  
cout << setprecision(2) << price; // Вывод: 1e+02 или 100
```

Правильно

```
cout << fixed << setprecision(2) << price; // 100.50
```

ПОЛЕЗНЫЕ СОВЕТЫ (ЧАСТЬ 1)

Советы по вводу-выводу

1. Всегда проверяйте успешность ввода. Например: ¹
2. Используйте `endl` только когда нужно сбросить буфер. Например: ²
3. Для таблиц используйте `setw`. Например: ³

ПОЛЕЗНЫЕ СОВЕТЫ (ЧАСТЬ 2)

Советы по переменным

1. 1
2. 2
3. 3

КОНТРОЛЬНЫЕ ВОПРОСЫ

10 вопросов для самопроверки:

1. В чем разница между `cin` и `cout`?
2. Что делают операторы `<<` и `>>`?
3. Как вывести число с 2 знаками после запятой?
4. В чем разница между `\n` и `endl`?
5. Как прочитать строку с пробелами?
6. Что такое литерал? Приведите примеры.
7. В чем разница между `'A'` и `"A"`?
8. Как объявить константу?
9. Что произойдет при делении $5/2$ в C++?
10. Как очистить буфер ввода после ошибки?

10 ответов:

1. `cin` — ввод данных, `cout` — вывод данных
2. `<<` — оператор вставки (вывод), `>>` — оператор извлечения (ввод)
3. `cout << fixed << setprecision(2) << number;`
4. `\n` — просто перенос, `endl` — перенос + сброс буфера
5. `getline(cin, string_variable);`
6. Литерал — фиксированное значение в коде (25, 3.14, `'A'`, `"text"`)
7. `'A'` — символ, `"A"` — строка
8. `const int MAX = 100;`
9. Результат будет 2 (целочисленное деление)
10. `cin.clear(); cin.ignore(32767, '\n');`

ШПАРГАЛКА (ЧАСТЬ 1)

Основные манипуляторы форматирования

Манипулятор	Описание	Пример
<code>endl</code>	Перенос строки + сброс буфера	<code>cout << endl;</code>
<code>\n</code>	Перенос строки	<code>cout << "\n";</code>
<code>\t</code>	Табуляция	<code>cout << "\t";</code>
<code>setw(n)</code>	Ширина поля	<code>cout << setw(10) << x;</code>
<code>setprecision(n)</code>	Точность	<code>cout << setprecision(2) << pi;</code>
<code>fixed</code>	Фиксированный формат	<code>cout << fixed << price;</code>
<code>scientific</code>	Научный формат	<code>cout << scientific << big;</code>
<code>left</code>	Выравнивание влево	<code>cout << left << text;</code>
<code>right</code>	Выравнивание вправо	<code>cout << right << text;</code>
<code>setfill('c')</code>	Символ заполнения	<code>cout << setfill('*') << setw(5) << 7;</code>

ШПАРГАЛКА (ЧАСТЬ 2)

Основные типы данных

Тип	Размер	Пример
int	4 байта	int age = 25;
double	8 байт	double pi = 3.14;
float	4 байта	float f = 3.14f;
char	1 байт	char c = 'A';
bool	1 байт	bool flag = true;
string	Зависит	string s = "text";

Специальные символы

Символ	Описание
\n	Перенос строки
\t	Табуляция
\\	Обратный слэш
\"	Двойная кавычка
\'	Одинарная кавычка
\0	Нулевой символ

ЗАКЛЮЧЕНИЕ

Вы изучили основы C++:

1. Потоки ввода-вывода (cin, cout)
2. Форматирование данных (iomanip)
3. Переменные и типы данных
4. Литералы и константы
5. Практические примеры (калькулятор, анкета, конвертер)
6. Частые ошибки и как их избежать
7. Полезные советы и лучшие практики

Следующие шаги:

1. Практикуйтесь: пишите свои программы
2. Экспериментируйте с разными типами данных
3. Изучите условные операторы (if, switch)
4. Изучите циклы (for, while, do-while)
5. Изучите функции и массивы
6. Изучите указатели и ссылки
7. Изучите объектно-ориентированное программирование (ООП)

Помните:

1. Ошибки — это нормально: они помогают учиться
2. Читайте сообщения об ошибках компилятора
3. Используйте отладчик (debugger)
4. Пишите чистый и понятный код
5. Комментируйте сложные части кода

Удачи в изучении C++!